Hazelcast - MapReduce I

MapReduce

MapReduce es un **modelo de programación** para el **procesamiento de grandes volúmenes de datos** de **manera paralela y distribuida** a partir de primitivas simples.

Este modelo, por sus características, está altamente ligado y fue piedra fundacional de la movida llamada Big Data

MapReduce

- Basado en un paper realizado por Dean & Ghemawat en Google
- El paper fue a partir de las conclusiones del trabajo del equipo de Google en la mejora del algoritmo Page Rank
- La necesidad era **poder procesar el gran volumen de información** en mejores tiempos
- El modelo se basa en
  - Las primitivas de programación funcional que le dan nombre al modelo (***map* y *reduce***)
  - En poder subdividir la data para su procesamiento **distribuido (*divide & conquer*)**
  - Trabajar con la información local al nodo lo más posible, evitando trasladarla por la red **(*Data Locality*)**

Datos

Para el modelo **cada unidad de información** o dato que se utiliza como entrada y salida de las etapas cuenta de dos partes:

- **una clave**: utilizada para clasificar o identificar la información
- **un valor:** con el contenido del dato

Cada uno puede ser tan complejo como cualquier objeto o registro de una base de datos, o tan simple como una primitiva

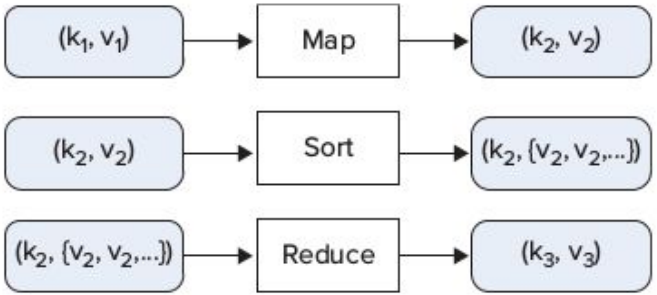
Entonces cada etapa será una función que, aplicada un par clave valor, retorna otro par clave valor:

```
[k2, v2] = F(k1, v1)
```

Etapas

Las dos operaciones se encargan de filtrar y transformar la data (***map***) para luego agregar esa *data* transformada para obtener el valor final (***reduce***)

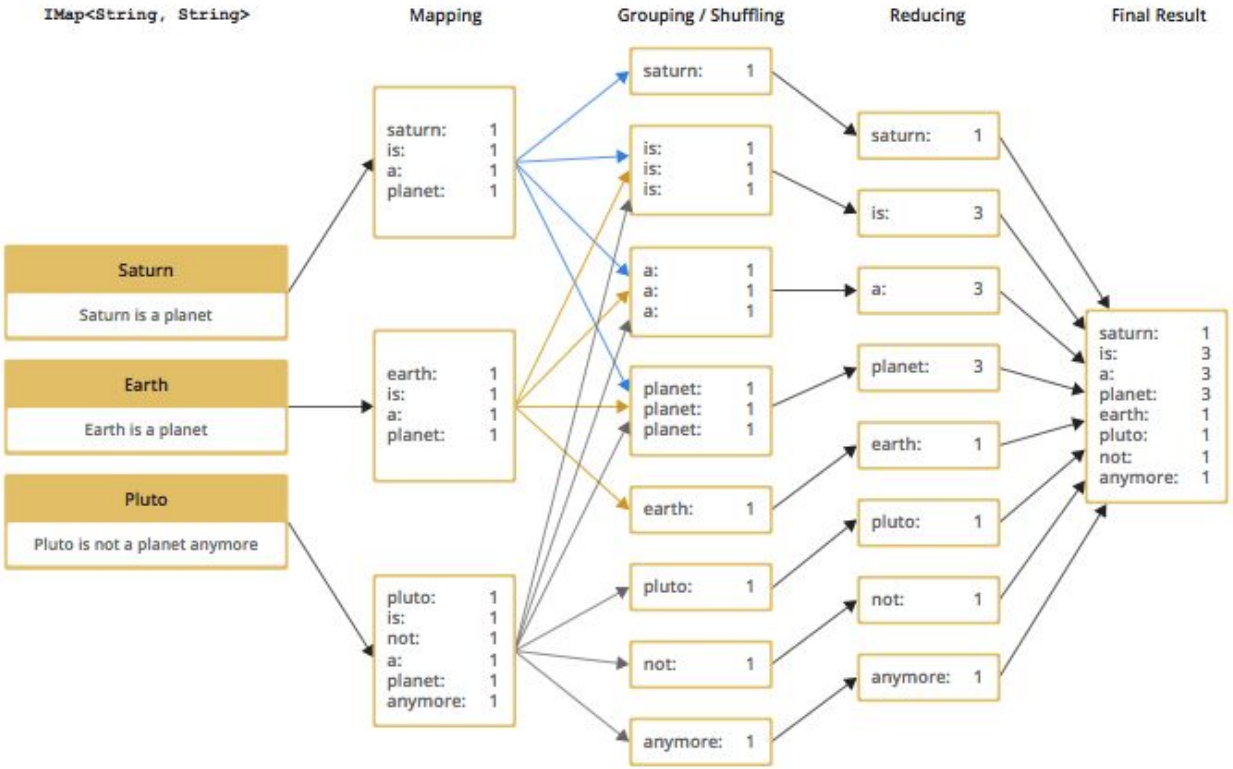
Entonces las etapas son:



- ***Map*** **y** ***Reduce*** **se programan**
- ***Sort/Shuffle*** **es provisto por el** ***framework***

Word Count

Vista de las etapas en el "Hello World"de Big Data: El Word Count



Map

La operación Map tiene por objetivo transformar la *data* inicial en información útil para la operación final. **Para lo cual:**

- **Filtra:** registros que no son necesarios no son emitidos
- **Projecta:** no emite valores que no son necesarios para la operación final
- **Expande:** agrega información que puede obtener de fuentes externas
- **Multiplica:** para ciertas operaciones tal vez se necesite emitir más de un valor
  Ejemplos:
  - dado un país -> emitir un valor por provincia
  - dado una frase -> emitir la palabra

Entonces la operación map:

- **recibe un par *(key, value)***
- **emite 0, 1 ó más pares *(key, value)***

Hazelcast - Mapper

La interfaz `Mapper` provee un método **map** que, por cada elemento del set de datos, **recibe** como parámetros **la clave, el valor del ítem, y una instancia de `Context` que permite emitir** los valores de salida

```
public interface Mapper<KeyIn, ValueIn, KeyOut, ValueOut> extends Serializable {

    void map(KeyIn key, ValueIn value, Context<KeyOut, ValueOut> context);

}
```

En caso de precisar acceder dentro del mapa a alguna otra clase o elemento de Hazelcast se puede implementar HazelcastInstanceAware.

Hazelcast - Mapper

Para el **WordCount**:

```
public class TokenizerMapper implements Mapper<String, String, String, Long> {

    private static final Long ONE = 1L;

    @Override
    public void map(String key, String value, Context<String, Long> context) {
        Pattern.compile("\\s+")
                .splitAsStream(value.toLowerCase())
                .forEach(word -> context.emit(word, ONE));
    }

}
```

Hazelcast - Mapper

- A cada *mapper* se le asigna una partición del nodo en que se ejecuta (si hay), para aprovechar *data locality*
- Si algún nodo tiene varias particiones podrían tener más de una instancia de *mapper*
- Si el *mapper* termina con su partición se le puede asignar una nueva del nodo
- Por cómo está armada la operación map es naturalmente *thread safe*

Sort

El *framework* se encarga de esta etapa por lo cual no requiere programación
Va a tomar cada par emitido por el *mapper* y juntar los valores correspondientes a la misma clave
Esto puede ser un punto complicado de implementación del *framework* porque requiere:

- Conservar los pares hasta que el *reducer* pueda consumir los valores
- Transmitir los valores por red para el nodo que está consumiendo la clave en cuestión

El *sort* se encarga de agrupar por *key* todos los valores emitidos por los *mappers* para enviarlos a cada *reducer*

Reduce

Se crea (o reutiliza) **un *reducer* por cada clave emitida**

El mismo **recibe todos los valores** emitidos **por todos los *mappers*** para la **misma clave**

A partir de esos valores el *reducer* los “procesa” para generar un “par final” de clave y valor (o valores)

Entonces la salida de cada *reducer* es un valor o valores para la clave que le fue asignada

El *reduce* transforma todos los valores emitidos para una clave y los transforma en un valor final

Hazelcast - Reducer

```
public interface ReducerFactory<KeyIn, ValueIn, ValueOut> extends Serializable {

   Reducer<ValueIn, ValueOut> newReducer(KeyIn key);

}
```

```
public abstract class Reducer<ValueIn, ValueOut> {

    public void beginReduce() {
    }

    public abstract void reduce(ValueIn value);

    public abstract ValueOut finalizeReduce();
}
```

Para implementar un *reducer* se deben implementar dos clases asociadas:

`ReducerFactory` y `Reducer`.

Hazelcast - Reducer

Para el **WordCount**:

```
public class WordCountReducerFactory implements ReducerFactory<String, Long, Long> {
    @Override
    public Reducer<Long, Long> newReducer(String key) {
        return new WordCountReducer();
    }
    private static class WordCountReducer extends Reducer<Long, Long> {
        private long sum;
        @Override
        public void reduce(Long value) {
            sum += value;
        }
        @Override
        public Long finalizeReduce() {
            return sum;
        }
    }
}
```

Hazelcast - Reducer

- Se requiere un *factory* porque se ejecuta **un *reducer* por cada clave única** emitida por los *mapper* (La clave es el parámetro del método `newReducer`).
- El ciclo de vida de un *reducer* es:
  - Al **llegar una clave** que aún no fue asignada a un *reducer* se utiliza **el factory para instanciar al nuevo *reducer***
  - Se **llama al método `beginReduce`** una vez
  - **Por cada valor** asociado a la clave **se invoca al método `reduce`**
  - Un vez que se terminaron los valores se invoca a **`finalizeReduce` para obtener el valor final para dicha clave**

Hazelcast - Reducer

- Como se envía de a *chunks*, los *reducers* pueden llegar a quedar suspendidos hasta que aparezcan nuevos valores para la clave asociada
- En el caso de una suspensión probablemente no se reinicie en el mismo *thread*. Antes de la versión 3.3 no había garantías de visibilidad de variables entre los *threads* por lo que había que definir todas las variables de instancia como *volatile*

Hazelcast - Job Tracker

Hazelcast provee una implementación de MapReduce además de otras herramientas de procesamiento y *query* distribuido

La API de Hazelcast para MapReduce está armada a partir de un DSL, que permite describir la tarea mediante el patrón Builder

El punto de entrada es la clase `JobTracker`, la misma se obtiene a partir de la instancia de Hazelcast (sea un *client node* o un *member node*)

```
ClientConfig clientConfig = ...;
HazelcastInstance hazelcastInstance = HazelcastClient.newHazelcastClient(clientConfig);

JobTracker jobTracker = hazelcastInstance.getJobTracker("word-count");
```

Hazelcast - Key Value Source

Un *job* obtiene la información a procesar mediante a la clase **`KeyValueSource`**, que permite al *job* iterar sobre los valores de una colección

La clase **`KeyValueSource`** provee métodos estáticos para construirla a partir de las colecciones distribuidas (`IMap`, `IList`, `ISet`, `Multimap`, etc.).

```
IMap<String, String> wordsIMap = hazelcastInstance.getMap("words");
KeyValueSource<String, String> wordsKeyValueSource = KeyValueSource.fromMap(wordsIMap);

IList<String> myList = hazelcastInstance.getList("my-list");
KeyValueSource<String, String> listKeyValueSource = KeyValueSource.fromList(myList);
```

En caso de colecciones sin clave aparente (o sea todo lo que no sea mapas), la clave es el nombre de la colección.

Hazelcast - Creación del Job

Una vez obtenida la `KeyValueSource` y definidos los *mappers* y *reducers* se utiliza el `JobTracker` para instanciar un Job, que funciona como Builder para setear los diferentes elementos del job y poder "*submitear*" el mismo

```
Job<String, String> job = jobTracker.newJob(wordsKeyValueSource);
ICompletableFuture<Map<String, Long>> future = job
      .mapper(new TokenizerMapper())
      .reducer(new WordCountReducerFactory())
      .submit();
```

```
// Wait and retrieve the result
Map<String, Long> result = future.get();
```

Resultado obtenido por vía sincrónica

Hazelcast - Creación del Job

```
Job<String, String> job = jobTracker.newJob(wordsKeyValueSource);
ICompletableFuture<Map<String, Long>> future = job
      .mapper(new TokenizerMapper())
      .reducer(new WordCountReducerFactory())
      .submit();
```

Resultado obtenido por vía asincrónica

```
// Attach a callback listener
future.andThen(new ExecutionCallback<>() {
   @Override
   public void onResponse(Map<String, Long> response) {
      ...
   }
   @Override
   public void onFailure(Throwable t) {
      logger.error("Error: {}", t.getMessage());
   }
});
```

Hazelcast - Ejecución

Para ejecutar un *job* en un *cluster* se requiere que cada nodo tenga las clases necesarias para ejecutarlas en el `CLASSPATH` (no hay carga dinámica de clases):

- *Mapper*
- *Reducer*
- Objetos que se utilicen como *keys* y/o *values*
- Otros objetos de apoyo para el *job*

Todos los nodos deben tener el mismo `hazelcast.xml` o la misma configuración programática para que formen el *cluster*

El cliente puede ser parte del *cluster* o no, depende de cómo se instancia la **`HazelcastInstance`**

Ejercicio 1

```
sh run-client.sh mobydick.txt
...
to=4421
a=4507
and=6141
of=6391
the=13999
```

Implementar el Job Map Reduce Word Count

- Clonar el *template repo* **https://github.com/POD-ITBA/word-count/**
- Cambiar **l12345** por su legajo para utilizar su propio cluster
- **En Client descomentar las últimas líneas**
- **En Api implementar** las clases **TokenizerMapper** y **WordCountReducerFactory**
- Ejecutar el cliente con un *cluster* de por lo menos dos nodos (dos terminales corriendo el **run-node.sh** y una corriendo el **run-client.sh**)

Ejercicio 2

```
sh run-client.sh mobydick.txt
...
a=apprehensiveness
b=bibliographical
c=characteristically
d=demonstrations
e=embellishments
```

Implementar un segundo Job Map Reduce

que utiliza el mismo *key-value source* anterior pero ahora se pide obtener:

- **Para cada letra, la palabra más larga que comienza con esa letra.**
- En el caso de que para una letra existan dos palabras con la misma longitud, quedarse con la menor alfabética.
- Sólo se deben considerar las palabras que tengan letras del alfabeto inglés (puede usar `word.matches("[a-z]+")`)

Takeaways

MapReduce se puede utilizar como alternativa de SQL donde:

- El *Mapper* realiza:
  - ➢ Una proyección equivalente al SELECT
  - ➢ Un filtrado equivalente al WHERE
- El sort/grouping/shuffling equivale al GROUP BY
- El *Reducer* realiza:
  - ➢ Una proyección: idem anterior, pero definitivo
  - ➢ Funciones de agregación
  - ➢ Un filtrado equivalente al HAVING

No son posibles ordenamientos globales de los resultados dentro del modelo de MapReduce ya que el reducer emite por clave

Esto último y alguna *queries* más complejas se hacen mediante un *post* procesamiento o con un segundo MapReduce posterior encadenado a los resultados del primero

Referencias y para profundizar

- Paper MapReduce
- Documentación Hazelcast Map Reduce

CREDITS

Content of the slides:

- POD - ITBA

Images:

- POD - ITBA
- Or obtained from: commons.wikimedia.org

Slides theme credit:

Special thanks to all the people who made and released these awesome resources for free:

- Presentation template by SlidesCarnival
- Photographs by Unsplash